Departamento de Computación
FCEFQyN, Universidad Nacional de Río Cuarto
Asignatura: Análisis y Diseño de Algoritmos I/Diseño de Algoritmos/Algoritmos II
Segundo Cuatrimestre de 2025

Examen Recuperatorio No. 2

Este examen debe ser resuelto en forma individual. No olvide poner su nombre y número de documento en el encabezado de la resolución.

1. Una empresa de logística debe distribuir productos dentro de cajas de envío. Cada caja soporta un *máximo de N unidades de peso* (con N un número real positivo). Se dispone de un conjunto de productos $e_1, e_2, \ldots, e_k$ cuyos pesos son $n_1, n_2, \ldots, n_k$ respectivamente.

   El objetivo es agrupar los productos en cajas de modo que:

   - La suma del peso de los productos dentro de cada caja **no exceda** la capacidad N.
   - Se utilice la **menor cantidad posible de cajas**.

   Diseñe e implemente en **Java**, un algoritmo que resuelva este problema utilizando la técnica *Greedy*, posiblemente de manera aproximada, y explicque claramente la estrategia empleada.

2. Una fábrica recibe diariamente una lista de pedidos que puede producir. Cada pedido i requiere t_i horas-máquina para ser fabricado (con t_i un número natural positivo) y otorga una ganancia asociada g_i (también un número natural positivo). La fábrica dispone de un total de T horas-máquina para la jornada.

   Debido a esta restricción, no es posible producir todos los pedidos disponibles. Se desea seleccionar un subconjunto de pedidos para maximizar la ganancia total obtenida, sin superar el límite de horas-máquina disponibles T.

   Diseño, e implemente en java, un algoritmo que, dados T, $(t_1, t_2, \ldots, t_n)$ y $(g_1, g_2, \ldots, g_n)$, calcule la *máxima ganancia* que puede obtenerse respetando las horas de producción disponibles. Su solución debe utilizar *Programación Dinámica*.

3. El *coeficiente de permutación* permite calcular la cantidad de secuencias ordenadas que se pueden construir a partir de un conjunto finito. Concretamente $P(n, k)$ representa el número de secuencias ordenadas de tamaño k que se pueden construir a partir de un conjunto de n elementos. El coeficiente de permutación cumple con la siguiente ecuación de recurrencia:

$$\begin{aligned} P(n,0) &= 1 \\ P(n,1) &= n \\ P(n,n) &= 1 \\ P(n,k) &= P(n-1,k) + k \times P(n-1,k-1) \end{aligned}$$

   Diseñe e implemente en Java, un programa que, dados n y k, compute en tiempo polinomial el coeficiente de permutación $P(n, k)$.

4. Considere el problema de, dada una multiplicación de m números naturales, computar el número total de formas de parentizar la multiplicación. Por ejemplo, para la multiplicación:

$$2 \times 3 \times 4 \times 5$$

existen la siguientes formas de parentización:

$$((2 \times 3) \times 4) \times 5, (2 \times (3 \times 4)) \times 5, (2 \times 3) \times (4 \times 5), 2 \times (3 \times (4 \times 5)), 2 \times ((3 \times 4) \times 5).$$

Siendo $n = m - 1$ el número de multiplicaciones involucradas, el siguiente programa calcula el número de parentizaciones:

```
public long parens(int n) {
    if (n <= 1) return 1;
    long res = 0;
    for (int i=0; i<n; i++)
        res += parens(i)*parens(n-i-1);
    return res;
}
```

Esta solución es exponencial en tiempo. Use *memoization* para llevar este programa a tiempo polinomial.